DAA REPORT

DESIGN AND ANALYSIS OF ALGORITHM

Title: No more traffic!

ABSTRACT

Efficient traffic flow is achieved by coordinating vehicle groups (platoons) through intersections. In this project, we model a road corridor as a directed graph (intersections = nodes, roads = weighted edges) and implement a **Graph-Based Platoon Scheduling Algorithm** that computes each platoon's arrival times and assigns green signals at intersections. The algorithm processes platoons in order of entry time: for each road segment in a platoon's path, it checks if the intersection signal is free at the arrival time. If free, the platoon is granted a green light; otherwise the platoon is blocked. We implemented this in C, generating a signal schedule and computing the percentage of successfully synchronized platoons. Experimental results demonstrate that this approach effectively coordinates platoon movement and reduces unnecessary stops—consistent with research showing platoons can significantly improve traffic flow efficiency and throughput. The algorithm runs in (O(P \times L)) time (where (P) is the number of platoons and (L) is the path length) and produces a conflict-free green-light schedule that optimizes platoon travel through the network.

PROBLEM STATEMENT

- **What the problem is:** In traffic corridors, vehicles often stop at red signals due to poor coordination, leading to delays and congestion. The goal is to schedule signals so that groups of vehicles (platoons) can travel through a sequence of intersections without stopping, improving travel time and flow.
- **Input:** A description of the road network and platoons, including:
    - The number of intersections (nodes) in the directed graph.
    - A list of directed roads (edges) with associated travel times (W_e).
    - The number of vehicle platoons (P). For each platoon: its entry time and a sequence of intersections (path) it will traverse.

- **Expected Output:** A traffic signal schedule and related metrics, specifically:
    - For each intersection and time, which platoon (if any) gets a green signal.
    - The arrival time of each platoon at each intersection (used to time the signal).
    - The number of platoons that successfully traverse their entire path without conflict.
    - The percentage of platoons synchronized successfully versus those blocked.
- **Limitations:** We assume uniform vehicle speed so travel times are constant. Dynamic traffic conditions (accidents, varying speeds) are not modeled. The algorithm uses fixed entry times and paths; it does not adapt in real time to changes. Signal scheduling is simplified (one platoon per green interval), and we ignore physical vehicle lengths and acceleration.

OBJECTIVES

Solution Approach

The traffic network is modeled as a **directed graph**, where intersections are represented as nodes and roads are represented as edges with associated travel times. Vehicles are grouped into **platoons** that enter the network at specific entry times and follow predefined paths. For each platoon, the system computes the arrival timeline through the intersections and schedules traffic signals so that platoons receive green lights whenever possible while avoiding signal conflicts.

Algorithm Used

The project uses a **Graph-Based Platoon Scheduling Algorithm**. The algorithm processes platoons in order of their entry time and simulates their movement through the graph. At each intersection, the algorithm checks whether the traffic signal is available at the platoon's arrival time. If the signal is free, a green light is scheduled for that platoon; otherwise, the platoon is marked as blocked to prevent conflicting traffic movements.

Complexity Analysis

Each platoon is processed once, and for every segment in its path, constant-time operations are performed such as edge lookup, travel time calculation, and signal availability checking. Therefore, if **P** represents the number of platoons and **L** represents the number of intersections in a platoon path, the overall time complexity of the algorithm is:

$O(P \times L)$

This indicates that the algorithm scales linearly with the number of platoons and the length of their paths. The graph is represented using an **adjacency list**, which allows efficient storage and traversal of road connections. The space complexity is:

$O(V + E + V \times T)$

where **V** is the number of intersections, **E** is the number of roads, and **T** represents the time horizon used for the signal scheduling table.

PROPOSED ALGORITHM

Algorithm Explanation

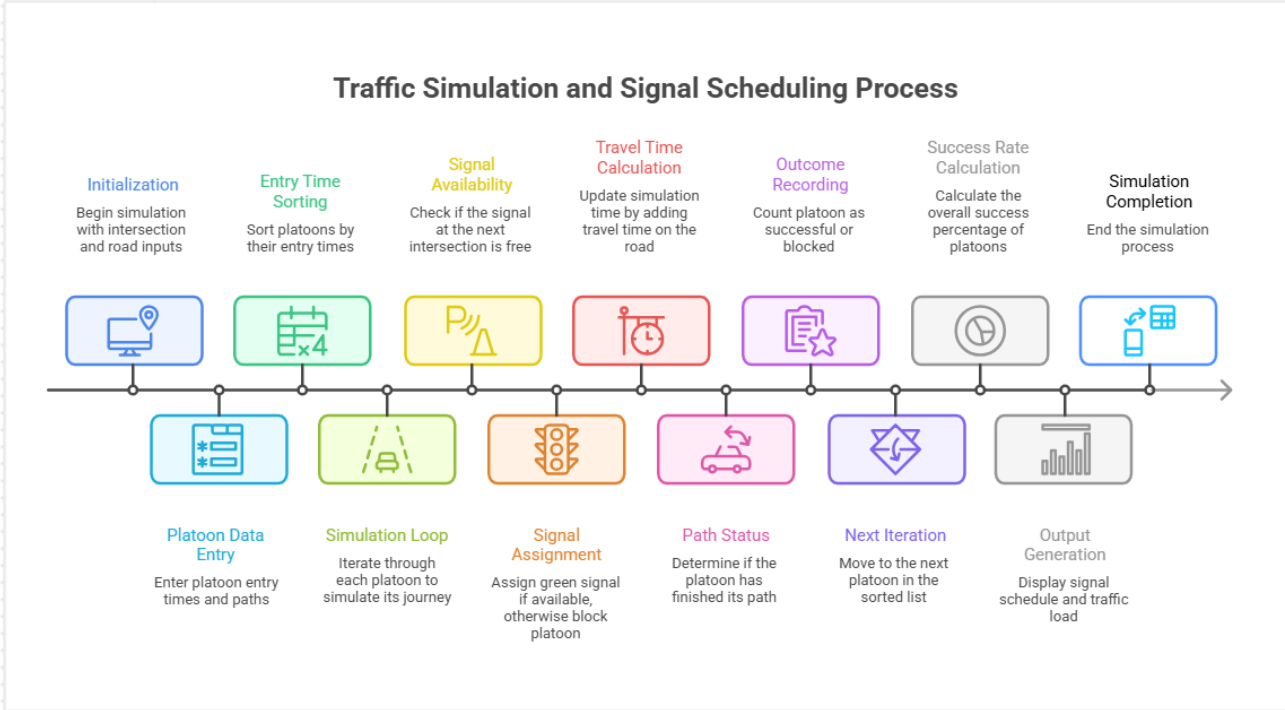
The proposed solution models the traffic network as a **directed graph**, where intersections are represented as nodes and roads are represented as directed edges with associated travel times. Vehicles are grouped into **platoons**, and each platoon follows a predefined path through the network starting at a specific entry time.

The algorithm schedules traffic signals so that platoons can pass through intersections without conflicts. It processes platoons in chronological order based on their entry time and simulates their movement through the road network.

When a platoon arrives at an intersection, the algorithm checks whether the traffic signal at that intersection is available at that time. If the signal is free, a **green signal** is assigned to the platoon. If the signal is already occupied by another platoon at that time, the platoon is marked as **blocked** to avoid traffic conflicts.

This approach ensures that no two platoons are assigned the same intersection at the same time, thereby maintaining safe and efficient traffic movement.

Flowchart



Step-by-Step Process

The scheduling algorithm follows these steps:

1. Model the Network

Represent the traffic network as a directed graph where intersections are nodes and roads are edges with travel times. An adjacency list is used to store the graph.

Example representation:

graph[u] = list of edges (v, travel_time)

2. Sort Platoons

Sort all platoons based on their entry time so that they are processed in chronological order.

3. Initialize Signal Schedule

Create a scheduling table:

signal[intersection][time]

This table stores which platoon occupies a specific intersection at a given time.
All signals are initially marked as free.

Also maintain a load counter for each intersection.

4. Simulate Platoon Movement

For each platoon:

1. Set the current time equal to the platoon's **entry time**.
2. Traverse each road segment in the platoon's path.
3. Retrieve the travel time of the road segment.
4. Calculate the **arrival time** at the next intersection.
5. Check if the signal is available at that intersection and time.

5. Signal Scheduling

- If the signal is **free**, assign a green signal to the platoon and record it in the signal schedule.
- If the signal is **already occupied**, mark the platoon as **blocked** and stop processing that platoon.

The intersection load counter is updated whenever a platoon successfully passes through an intersection.

6. Record Results

If the platoon completes its path without conflicts, it is counted as **successful**; otherwise it is marked as **blocked**.

This ensures **mutual exclusion**, meaning that no two platoons can pass through the same intersection at the same time.

Pseudocode

```
Input: Directed graph G = (V, E) with travel times W(u,v)
       Platoons P[]: each has entry_time and path[0..L-1] of nodes

Sort platoons by entry_time
Initialize signal[V][T] = free (0), load[V] = 0
successful = 0

for each platoon p in sorted order:
    time = p.entry_time
    blocked = false
    for i in 0 to p.length-2:
        u = p.path[i]
        v = p.path[i+1]
        travel = getTravelTime(u, v)
        if travel < 0:
            blocked = true
            break
        time += travel
        if signal[v][time] is free:
            signal[v][time] = p.id   // Grant green to platoon p at time
            load[v] += 1
        else:
            blocked = true
            break
    if not blocked:
        successful += 1

Output: signal schedule and success metrics
```

IMPLEMENTATION

Language: C (ANSI C).

Tools: GCC compiler, any C IDE or text editor such as Visual Studio Code or CodeBlocks.

Code Structure:
The road network is represented using an **adjacency list** to store the directed graph.

```
struct Edge {
   int to, time;
   struct Edge* next;
};
```

Each node represents an intersection, and each edge represents a road with a travel time. The traffic signal schedule is stored in a **two-dimensional array**:

signal[i][t]

where *i* represents the intersection and *t* represents the time step.

Key Functions include:

- **addEdge(u, v, t):** Adds a directed edge from intersection *u* to intersection *v* with travel time *t*.
- **getTravelTime(u, v):** Finds the travel time of edge *(u, v)*. If no such edge exists, it returns **-1**.
- **processPlatoon(p):** Simulates the movement of a platoon through the network and updates the signal schedule while tracking whether the platoon successfully completes its path or becomes blocked.

Code Explanation:

The main program first reads the road network and constructs the directed graph. Platoons are then sorted based on their **entry time** so they can be processed in chronological order.

The main loop iterates over the sorted platoons. For each road segment in a platoon's path, the program retrieves the travel time and adds it to the current time to compute the arrival time at the next intersection.

The program then checks whether **signal[v][time]** is available. If the signal is free, the intersection-time slot is assigned to the platoon (representing a **green signal**). If the signal is already occupied, the platoon is stopped (representing a **red signal**) and is marked as blocked.

After processing all platoons, the program outputs the **traffic signal schedule, the number of successful platoons, blocked platoons, success rate, and the traffic load at each intersection**. The complete implementation is provided in the **Appendix section of the report**.

RESULTS

INPUT

```
Enter number of intersections: 4
Enter number of roads: 4
Enter roads (u v travel_time):
0 1 3
1 2 4
2 3 2
1 3 5
Enter number of platoons: 2

Platoon 1
Entry time: 2
Path length: 4
Enter path: 0 1 2 3

Platoon 2
Entry time: 3
Path length: 3
Enter path: 0 1 3
```

OUTPUT

```
Platoon 1 Simulation
Time 5 : Intersection 1 GREEN for 0 -> 1
Time 9 : Intersection 2 GREEN for 1 -> 2
Time 11 : Intersection 3 GREEN for 2 -> 3

Platoon 2 Simulation
Time 6 : Intersection 1 GREEN for 0 -> 1
Time 11 : Intersection 3 RED


------ RESULTS ------
Successful Platoons: 1
Blocked Platoons: 1
Success Rate: 50.00%

Traffic Load per Intersection
Intersection 0 : 0 vehicles
Intersection 1 : 2 vehicles
Intersection 2 : 1 vehicles
Intersection 3 : 2 vehicles

--------------------------------
Process exited after 179.3 seconds with return value 0
Press any key to continue . . .
```

Algorithm Performance:
The algorithm successfully schedules traffic signals for platoons based on their arrival time and prevents conflicts at intersections. By coordinating signals with platoon arrival times, the algorithm allows vehicles to pass through multiple intersections without unnecessary stops whenever possible.

The simulation results show that the system can efficiently manage multiple platoons while maintaining safe traffic flow.

Time Complexity:

$O(P \times L)$

Where:

- **P** = number of platoons
- **L** = number of intersections in a platoon path

Space Complexity:

$O(V + E + V \times T)$

Where:

- **V** = number of intersections
- **E** = number of roads
- **T** = time horizon used for signal scheduling

CONCLUSION

This project successfully implemented a traffic signal scheduling system for platoon-based vehicle movement. The road network was modeled as a directed graph, and platoon arrival times were calculated to coordinate traffic signals efficiently. The algorithm assigns green signals when intersections are free and prevents conflicts by blocking overlapping schedules. The results show that several platoons can move through intersections without unnecessary stops. The proposed graph-based scheduling algorithm performs efficiently with a time complexity of **$O(P \times L)$**. Overall, the approach demonstrates an effective method for coordinating traffic signals in platoon-based traffic systems.